QUESTION 1: Employee Compensation Classification

Table Structure

```
CREATE TABLE employee_payments (

emp_id INT PRIMARY KEY,

emp_name VARCHAR(50),

department VARCHAR(30),

base_salary DECIMAL(10,2),

bonus DECIMAL(10,2),

joining_date DATE

);
```

Insert Data

```
INSERT INTO employee_payments VALUES

(1,'karthik','Data',75000.75,5000.50,'2019-03-15'),

(2,'veena','HR',65000.40,4000.25,'2021-06-20'),
```

(3,'ravi','Data',85000.90,6000.75,'2016-01-10'),

(4,'anil','Finance',70000.10,NULL,'2020-09-01'),

(5,'suresh','HR',60000.55,3000.30,'2022-11-25');

Question

For each employee:

· Convert emp_name to proper case ---upper /lower ---Initcap (CamelCase)

· Calculate total income = base_salary + bonus (NULL safe) +

· Round total income to nearest integer

· Extract joining year

· Use CASE to classify:

o Senior if experience > 7 years

o Mid if between 4 and 7

o Junior otherwise

QUESTION 2: Order Delivery Delay Analysis

Table Structure

```
CREATE TABLE orders_delivery (

order_id INT,

customer_name VARCHAR(50),

order_date DATE,

delivery_date DATE,

order_amount DECIMAL(10,2)

);
```

Insert Data

```
INSERT INTO orders_delivery VALUES

(101,'rajesh','2025-01-01','2025-01-05',12500.75),

(102,'meena','2025-01-10','2025-01-10',8400.40),
```

(103,'arun','2025-01-15','2025-01-20',15600.90),

(104,'pooja','2025-01-18',NULL,9200.10);

Question

For each order:

· Uppercase customer name

· Calculate delivery days using date difference

· Replace NULL delivery date with today

· Truncate order amount to 1 decimal

· Use CASE:

o Same-day

o Delayed (>3 days)

o Pending

QUESTION 3: Customer Spending Pattern

Table Structure

```
CREATE TABLE customer_spending (

cust_id INT,

cust_name VARCHAR(50),

city VARCHAR(30),

purchase_amount DECIMAL(10,2),

purchase_date DATE

);
```

Insert Data

```
INSERT INTO customer_spending VALUES

(1,'amit','mumbai',12000.75,'2024-12-01'),

(2,'neha','delhi',8500.40,'2024-12-15'),

(3,'rohit','mumbai',15500.90,'2024-11-20'),

(4,'kavya','chennai',6000.10,'2024-10-05');
```

Question

Display:

· Customer name with first letter capitalized

· Month name of purchase

· Rounded purchase amount

· Absolute value of purchase (defensive logic)

· CASE:

o High spender > 15000

o Medium 8000–15000

o Low otherwise

QUESTION 4: Subscription Validity Check

Table Structure

CREATE TABLE subscriptions (

```
user_id INT,

user_email VARCHAR(100),

start_date DATE,

end_date DATE,

subscription_fee DECIMAL(10,2)

);
```

Insert Data

```
INSERT INTO subscriptions VALUES

(1,'karthik@gmail.com','2024-01-01','2025-01-01',12000.50),

(2,'veena@yahoo.com','2024-06-15','2024-12-15',8500.75),

(3,'ravi@hotmail.com','2023-03-01','2024-03-01',15000.90);
```

Question

For each user:

· Extract email domain

· Calculate subscription duration in months

· Format fee with commas

· Find remaining days from today

· CASE:

o Active

o Expiring Soon (≤30 days)

o Expired

QUESTION 5: Loan EMI Risk Categorization

Table Structure

CREATE TABLE loan_details (

loan_id INT,

customer_name VARCHAR(50),

```
loan_amount DECIMAL(12,2),

interest_rate DECIMAL(5,2),

loan_start DATE

);
```

Insert Data

```
INSERT INTO loan_details VALUES

(201,'suresh',500000.75,8.5,'2022-01-10'),

(202,'mahesh',750000.40,9.2,'2021-05-20'),

(203,'anita',300000.90,7.8,'2023-07-01');
```

Question

Compute:

· Monthly interest using power function

· Years since loan start

· Round EMI

· Uppercase customer name

· CASE:

o High Risk if interest > 9

o Medium Risk

o Low Risk

QUESTION 6: Employee Attendance Evaluation

Table Structure

CREATE TABLE attendance (

emp_id INT,

emp_name VARCHAR(50),

total_days INT,

present_days INT,

record_date DATE

);

Insert Data

INSERT INTO attendance VALUES

(1,'karthik',30,28,'2025-01-31'),

(2,'veena',30,22,'2025-01-31'),

(3,'ravi',30,18,'2025-01-31');

Question

Calculate:

· Attendance percentage (rounded)

· Month name

· Difference between total and present days

· Lowercase employee name

· CASE:

o Excellent ≥90%

o Average 75–89%

o Poor otherwise

QUESTION 7: Product Discount Validation

Table Structure

```
CREATE TABLE product_sales (

product_id INT,

product_name VARCHAR(50),

mrp DECIMAL(10,2),

selling_price DECIMAL(10,2),

sale_date DATE

);
```

Insert Data

```
INSERT INTO product_sales VALUES

(1,'Laptop',75000.75,68000.50,'2025-01-10'),

(2,'Mobile',35000.40,33000.25,'2025-01-12'),

(3,'Tablet',25000.90,26000.75,'2025-01-15');
```

Question

Derive:

· Discount amount (absolute)

· Discount percentage

· Day name of sale

· Proper case product name

· CASE:

o Valid Discount

o Overpriced

o No Discount

QUESTION 8: Insurance Policy Aging

Table Structure

```
CREATE TABLE insurance_policies (

policy_id INT,

holder_name VARCHAR(50),

premium_amount DECIMAL(10,2),

policy_start DATE,

policy_end DATE

);
```

Insert Data

```
INSERT INTO insurance_policies VALUES

(301,'arjun',12000.50,'2023-01-01','2026-01-01'),

(302,'megha',8500.75,'2022-06-15','2025-06-15'),
```

(303,'vinod',15000.90,'2021-03-01','2024-03-01');

Question

Show:

· Policy duration in years

· Remaining days

· Rounded premium

· Uppercase holder name

· CASE:

o Long Term

o Mid Term

o Expired

QUESTION 9: Salary Increment Simulation

Table Structure

```
CREATE TABLE salary_revision (

emp_id INT,

emp_name VARCHAR(50),

current_salary DECIMAL(10,2),

rating INT,

last_hike DATE

);
```

Insert Data

```
INSERT INTO salary_revision VALUES

(1,'karthik',75000.75,5,'2023-01-01'),

(2,'veena',65000.40,4,'2024-01-01'),

(3,'ravi',85000.90,3,'2022-01-01');
```

Question

Calculate:

· Years since last hike

· Increment using rating logic

· New salary (rounded)

· Lowercase name

· CASE:

o High Increment

o Moderate

o No Increment

QUESTION 10: Customer Account Status Evaluation

Table Structure

CREATE TABLE bank_accounts (

account_id INT,

customer_name VARCHAR(50),

balance DECIMAL(12,2),

last_transaction DATE,

branch VARCHAR(30)

);

Insert Data

INSERT INTO bank_accounts VALUES

(501,'ramesh',125000.75,'2024-12-20','hyderabad'),

(502,'sita',8500.40,'2023-06-15','delhi'),

(503,'manoj',-2500.90,'2025-01-05','mumbai');

Question

Determine:

· Absolute balance

· Days since last transaction

· Proper case branch name

· Sign of balance

· CASE:

o Active

o Dormant

o Overdrawn

Level -1

QUESTION 1 – Salary Risk Flagging Based on Tax Shock

Table

CREATE TABLE salary_audit (

emp_id INT,

emp_name VARCHAR(50),

```
salary DECIMAL(10,2),

tax_percent DECIMAL(5,2),

last_revision DATE

);
```

Data

```
INSERT INTO salary_audit VALUES

(1,'karthik',75000.75,10.5,'2022-01-15'),

(2,'veena',65000.40,18.0,'2023-06-01'),

(3,'ravi',85000.90,25.0,'2020-11-20');
```

Question

For each employee:

· Normalize name to lowercase

· Calculate net salary after tax and round it

· Extract revision year

· Find months since revision

· Use CASE:

o Flag Tax Shock if tax > 20 AND months > 24

o Flag Review Needed if tax between 15–20

o Else Stable

QUESTION 2 – Bonus Abuse Detection

Table

```
CREATE TABLE bonus_monitor (

emp_code INT,

emp_name VARCHAR(50),

base_salary DECIMAL(10,2),

bonus DECIMAL(10,2),
```

```
bonus_date DATE

);
```

Data

```
INSERT INTO bonus_monitor VALUES

(101,'Anil',70000.10,30000.00,'2025-01-10'),

(102,'Suresh',60000.55,3000.30,'2024-03-15'),

(103,'Ravi',85000.90,15000.75,'2023-12-01');
```

Question

For each record:

· Convert name to proper case

· Calculate bonus percentage of salary (rounded)

· Extract day name of bonus

· Find absolute salary–bonus difference

· CASE:

o Suspicious if bonus > 30% AND weekend

o Normal if bonus <= 20%

o Audit

QUESTION 3 – Experience Parity Validation

Table

```
CREATE TABLE employee_experience (

emp_id INT,

emp_name VARCHAR(50),

joining_date DATE,

declared_experience INT,

salary DECIMAL(10,2)

);
```

Data

```
INSERT INTO employee_experience VALUES

(1,'Veena','2018-07-01',4,65000.40),

(2,'Ravi','2014-01-10',12,85000.90),

(3,'Anil','2020-09-01',3,70000.10);
```

Question

For each employee:

· Uppercase name

· Calculate actual experience from date

· Find difference between declared and actual experience

· Floor salary

· CASE:

o Overstated if declared > actual

o Understated if declared < actual

o Matched

QUESTION 4 – Salary Digit Pattern Analysis

Table

```
CREATE TABLE salary_digits (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

credit_date DATE

);
```

Data

```
INSERT INTO salary_digits VALUES

(1,'Karthik',75000.75,'2025-01-01'),

(2,'Veena',65000.40,'2025-01-02'),
```

(3,'Suresh',60000.55,'2025-01-03');

Question

For each employee:

· Extract last two characters of name

· Get day of month from credit date

· Truncate salary to integer

· Use MOD on salary

· CASE:

o Pattern Match if salary MOD 10 equals day

o No Match otherwise

QUESTION 5 – Odd–Even Salary Compliance

Table

CREATE TABLE payroll_control (

```
emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

payment_date DATE

);
```

Data

```
INSERT INTO payroll_control VALUES

(1,'Ravi',85000.90,'2025-01-15'),

(2,'Anil',70000.10,'2025-01-16'),

(3,'Veena',65000.40,'2025-01-17');
```

Question

For each employee:

· Lowercase name

· Extract weekday

· Round salary

· Apply MOD on salary

· CASE:

o Violation if even salary paid on odd weekday

o Compliant otherwise

QUESTION 6 – Salary Inflation Drift

Table

```
CREATE TABLE inflation_watch (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

last_hike DATE

);
```

Data

```
INSERT INTO inflation_watch VALUES

(1,'Karthik',75000.75,'2019-01-01'),

(2,'Veena',65000.40,'2022-01-01'),

(3,'Ravi',85000.90,'2017-01-01');
```

Question

For each employee:

· Proper case name

· Calculate years since hike

· Apply POWER on years

· Round salary impact

· CASE:

o High Inflation Risk if years > 5

o Moderate

o Low

QUESTION 7 – Salary Sign Integrity Check

Table

```
CREATE TABLE salary_integrity (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

record_date DATE

);
```

Data

```
INSERT INTO salary_integrity VALUES

(1,'Anil',-70000.10,'2025-01-10'),
```

(2,'Veena',65000.40,'2025-01-10'),

(3,'Ravi',0.00,'2025-01-10');

Question

For each employee:

· Uppercase name

· Extract year

· Apply SIGN on salary

· ABS salary

· CASE:

o Negative Error

o Zero Salary

o Valid

QUESTION 8 – Name Length vs Salary Correlation

Table

```
CREATE TABLE name_salary (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

join_date DATE

);
```

Data

```
INSERT INTO name_salary VALUES

(1,'Karthik',75000.75,'2019-03-15'),

(2,'Veena',65000.40,'2021-06-20'),

(3,'Ravi',85000.90,'2016-01-10');
```

Question

For each employee:

· Calculate name length

· Calculate years of service

· Round salary

· Compare name length vs years

· CASE:

o Name Bias if length > years

o Neutral

QUESTION 9 – Salary Spike Detection by Month

Table

CREATE TABLE salary_monthly (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

```
paid_date DATE

);
```

Data

```
INSERT INTO salary_monthly VALUES

(1,'Karthik',75000.75,'2025-01-31'),

(2,'Veena',65000.40,'2025-02-28'),

(3,'Ravi',85000.90,'2025-03-31');
```

Question

For each record:

· Extract month name

· CEIL salary

· Check last day of month

· CASE:

o End Month Spike

o Regular

QUESTION 10 – Salary Digit Sum Audit

Table

```
CREATE TABLE digit_audit (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

audit_date DATE

);
```

Data

```
INSERT INTO digit_audit VALUES

(1,'Anil',70000.10,'2025-01-01'),
```

(2,'Veena',65000.40,'2025-01-02');

Question

For each employee:

· Extract first character of name

· Truncate salary

· Sum digits logically

· Extract day

· CASE:

o Digit Alert

o Normal

QUESTION 11 – Weekend Salary Credit Fraud Detection

Table

CREATE TABLE salary_credit_audit (

```
emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

credit_date DATE,

bank_code VARCHAR(10)

);
```

Data

```
INSERT INTO salary_credit_audit VALUES

(1,'Karthik',75000.75,'2025-01-04','HDFC01'),

(2,'Veena',65000.40,'2025-01-06','ICIC02'),

(3,'Ravi',85000.90,'2025-01-05','SBIN03'),

(4,'Anil',70000.10,'2025-01-07','AXIS04'),

(5,'Suresh',60000.55,'2025-01-11','HDFC01');
```

Question

For each record:

· Extract bank prefix from bank_code

· Identify weekday name of credit_date

· Round salary

· Apply MOD on salary

· CASE:

o Weekend Fraud if credited on Saturday/Sunday AND salary MOD 5 = 0

o Bank Review if bank is HDFC

o Else Normal

QUESTION 12 – Salary Credit Time Drift Analysis

Table

CREATE TABLE salary_time_drift (

emp_id INT,

```
emp_name VARCHAR(50),

salary DECIMAL(10,2),

credit_ts DATETIME

);
```

Data

```
INSERT INTO salary_time_drift VALUES

(1,'Karthik',75000.75,'2025-01-10 23:45:00'),

(2,'Veena',65000.40,'2025-01-10 09:15:00'),

(3,'Ravi',85000.90,'2025-01-11 00:10:00'),

(4,'Anil',70000.10,'2025-01-09 18:30:00'),

(5,'Suresh',60000.55,'2025-01-10 02:50:00');
```

Question

For each employee:

· Extract hour from credit timestamp

· Convert emp_name to lowercase

· Floor salary

· Calculate difference between salary and hour

· CASE:

o Midnight Drift if hour between 0–3

o After Hours

o Business Hours

QUESTION 13 – Salary Decimal Precision Audit

Table

CREATE TABLE salary_precision (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,4),

record_date DATE

);

Data

INSERT INTO salary_precision VALUES

(1,'Karthik',75000.7567,'2025-01-01'),

(2,'Veena',65000.4044,'2025-01-02'),

(3,'Ravi',85000.9099,'2025-01-03'),

(4,'Anil',70000.1001,'2025-01-04'),

(5,'Suresh',60000.5555,'2025-01-05');

Question

For each record:

· Truncate salary to 2 decimals

· Calculate difference between rounded and truncated value

· Extract day name

· Get length of emp_name

· CASE:

o Precision Loss if difference > 0.01

o Safe

QUESTION 14 – Salary Growth Power Index

Table

CREATE TABLE salary_growth (

emp_id INT,

emp_name VARCHAR(50),

base_salary DECIMAL(10,2),

growth_rate DECIMAL(5,2),

last_hike DATE

);

Data

INSERT INTO salary_growth VALUES

(1,'Karthik',75000.75,1.08,'2019-01-01'),

(2,'Veena',65000.40,1.05,'2021-01-01'),

(3,'Ravi',85000.90,1.12,'2017-01-01'),

(4,'Anil',70000.10,1.03,'2022-01-01'),

(5,'Suresh',60000.55,1.06,'2020-01-01');

Question

For each employee:

· Calculate years since last hike

· Apply POWER using growth_rate and years

· Round projected salary

· Uppercase emp_name

· CASE:

o Explosive Growth if projected > 150000

o Controlled

o Stagnant

QUESTION 15 – Salary Symmetry Check

Table

```
CREATE TABLE salary_symmetry (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

processed_date DATE

);
```

Data

```
INSERT INTO salary_symmetry VALUES

(1,'Karthik',75557.75,'2025-01-15'),

(2,'Veena',64446.40,'2025-01-16'),

(3,'Ravi',85858.90,'2025-01-17'),

(4,'Anil',70007.10,'2025-01-18'),

(5,'Suresh',60000.55,'2025-01-19');
```

Question

For each record:

· Remove decimals from salary

· Reverse salary digits

· Extract weekday

· Proper case emp_name

· CASE:

o Symmetric Pay if reversed equals original

o Asymmetric

QUESTION 16 – Leap Year Salary Adjustment Audit

Table

```
CREATE TABLE leap_salary (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

credit_date DATE

);
```

Data

```
INSERT INTO leap_salary VALUES

(1,'Karthik',75000.75,'2024-02-29'),
```

(2,'Veena',65000.40,'2025-02-28'),

(3,'Ravi',85000.90,'2020-02-29'),

(4,'Anil',70000.10,'2023-02-28'),

(5,'Suresh',60000.55,'2024-02-28');

Question

For each employee:

· Extract year

· Check leap year logic

· CEIL salary

· Calculate day of year

· CASE:

o Leap Credit

o Non-Leap Credit

QUESTION 17 – Fiscal Year Boundary Salary Check

Table

```
CREATE TABLE fiscal_salary (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

credit_date DATE

);
```

Data

```
INSERT INTO fiscal_salary VALUES

(1,'Karthik',75000.75,'2025-03-31'),

(2,'Veena',65000.40,'2025-04-01'),

(3,'Ravi',85000.90,'2024-03-30'),
```

(4,'Anil',70000.10,'2024-04-02'),

(5,'Suresh',60000.55,'2025-03-29');

Question

For each record:

· Determine fiscal year

· Extract month

· Format salary

· Lowercase emp_name

· CASE:

o Year End Credit

o Year Start Credit

o Mid Year

QUESTION 18 – Salary Random Sampling for Audit

Table

```
CREATE TABLE salary_sampling (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

record_date DATE

);
```

Data

```
INSERT INTO salary_sampling VALUES

(1,'Karthik',75000.75,'2025-01-01'),

(2,'Veena',65000.40,'2025-01-02'),

(3,'Ravi',85000.90,'2025-01-03'),

(4,'Anil',70000.10,'2025-01-04'),

(5,'Suresh',60000.55,'2025-01-05'),
```

(6,'Amit',72000.60,'2025-01-06'),

(7,'Neha',68000.80,'2025-01-07');

Question

For each record:

· Generate random value

· Round salary

· Extract day name

· Extract first character of emp_name

· CASE:

o Sampled if RAND() > 0.7

o Skipped

QUESTION 19 – Salary ASCII Integrity Check

Table

```
CREATE TABLE salary_ascii (

emp_id INT,

emp_name VARCHAR(50),

salary DECIMAL(10,2),

join_date DATE

);
```

Data

```
INSERT INTO salary_ascii VALUES

(1,'Karthik',75000.75,'2019-03-15'),

(2,'Veena',65000.40,'2021-06-20'),

(3,'Ravi',85000.90,'2016-01-10'),

(4,'Anil',70000.10,'2020-09-01'),

(5,'Suresh',60000.55,'2022-11-25');
```

Question

For each employee:

· Extract ASCII value of first character

· Calculate years since joining

· Floor salary

· Compare ASCII vs years

· CASE:

o Name Dominates

o Experience Dominates

QUESTION 20 – Salary vs Calendar Symmetry Logic

Table

CREATE TABLE salary_calendar (

emp_id INT,

```
emp_name VARCHAR(50),

salary DECIMAL(10,2),

credit_date DATE

);
```

Data

```
INSERT INTO salary_calendar VALUES

(1,'Karthik',75000.75,'2025-01-15'),

(2,'Veena',65000.40,'2025-02-14'),

(3,'Ravi',85000.90,'2025-03-31'),

(4,'Anil',70000.10,'2025-04-04'),

(5,'Suresh',60000.55,'2025-05-05');
```

Question

For each record:

· Extract day and month

· Extract last two digits of salary

· Uppercase emp_name

· Absolute difference between day and month

· CASE:

o Calendar Match if day = month OR salary digits match

o Calendar Drift

LEVEL -2

QUESTION 1 – Employee Login Discipline & Performance Classification

Table Structure

CREATE TABLE employee_login (

emp_id INT,

emp_name VARCHAR(50),

login_time DATETIME,

```
logout_time DATETIME

);
```

Insert Data

```
INSERT INTO employee_login VALUES

(1,'Karthik','2025-01-15 09:05:00','2025-01-15 18:10:00'),

(2,'Veena','2025-01-14 10:30:00','2025-01-14 16:00:00'),

(3,'Ravi','2025-01-13 09:00:00','2025-01-13 20:00:00'),

(4,'Anil','2025-01-12 11:00:00','2025-01-12 14:00:00'),

(5,'Suresh','2025-01-11 09:15:00','2025-01-11 17:00:00');
```

Question

For each employee:

· Convert emp_name to proper case

· Identify whether login date is Weekday or Weekend

· Calculate total working hours (logout – login)

· Round working hours to 2 decimals

· Use CASE:

o Good Performer if weekday AND working hours ≥ 8

o Bad Performer if weekday AND working hours < 6

o Weekend Login otherwise

QUESTION 2 – Past 7 Days Attendance & Productivity Check

Table Structure

```
CREATE TABLE attendance_log (

emp_id INT,

emp_name VARCHAR(50),

login_date DATE,

login_time TIME,
```

```
logout_time TIME

);
```

Insert Data

```
INSERT INTO attendance_log VALUES

(1,'Karthik','2025-01-14','09:00:00','18:00:00'),

(2,'Karthik','2025-01-13','09:15:00','17:30:00'),

(3,'Veena','2025-01-12','10:00:00','15:00:00'),

(4,'Ravi','2025-01-10','09:00:00','19:00:00'),

(5,'Anil','2025-01-08','11:00:00','14:00:00');
```

Question

For each record:

· Uppercase employee name

· Check if login_date falls within last 7 days from today

· Identify Weekday / Weekend

· Calculate working hours using TIMEDIFF

· Use CASE:

o Active & Productive if last 7 days AND hours ≥ 8

o Active but Low Hours if last 7 days AND hours < 8

o Absent from Last 7 Days

QUESTION 3 – Weekend Work Abuse Detection

Table Structure

```
CREATE TABLE weekend_monitor (

emp_id INT,

emp_name VARCHAR(50),

work_date DATE,

login_time TIME,

logout_time TIME
```

);

Insert Data

INSERT INTO weekend_monitor VALUES

(1,'Ravi','2025-01-11','09:00:00','21:00:00'),

(2,'Veena','2025-01-12','10:00:00','13:00:00'),

(3,'Karthik','2025-01-10','09:00:00','18:00:00'),

(4,'Anil','2025-01-09','11:00:00','14:00:00');

Question

For each employee:

· Extract day name from work_date

· Lowercase employee name

· Calculate working hours

· Apply CEIL on hours

· Use CASE:

o Weekend Overtime if Saturday/Sunday AND hours ≥ 8

o Suspicious Login if weekend AND hours < 4

o Normal Working Day

QUESTION 4 – Login Time Deviation & Discipline Score

Table Structure

```
CREATE TABLE login_discipline (

emp_id INT,

emp_name VARCHAR(50),

login_datetime DATETIME,

logout_datetime DATETIME

);
```

Insert Data

INSERT INTO login_discipline VALUES

(1,'Karthik','2025-01-15 08:55:00','2025-01-15 18:10:00'),

(2,'Veena','2025-01-15 10:45:00','2025-01-15 16:00:00'),

(3,'Ravi','2025-01-15 09:00:00','2025-01-15 20:30:00'),

(4,'Anil','2025-01-15 11:30:00','2025-01-15 14:00:00');

Question

For each employee:

· Extract login hour

· Calculate total working hours

· Truncate working hours to 1 decimal

· Get weekday name

· Use CASE:

o Disciplined if weekday AND login before 9 AND hours ≥ 8

o Late Comer if weekday AND login after 10

o Poor Discipline otherwise

QUESTION 5 – Absenteeism vs Performance Correlation

Table Structure

```
CREATE TABLE performance_tracker (

emp_id INT,

emp_name VARCHAR(50),

work_date DATE,

login_time TIME,

logout_time TIME

);
```

Insert Data

```
INSERT INTO performance_tracker VALUES

(1,'Karthik','2025-01-09','09:00:00','18:00:00'),
```

(2,'Karthik','2025-01-10','09:10:00','17:50:00'),

(3,'Veena','2025-01-05','10:00:00','15:00:00'),

(4,'Ravi','2025-01-14','09:00:00','19:00:00'),

(5,'Anil','2025-01-03','11:00:00','14:00:00');

Question

For each record:

· Identify whether work_date is within last 7 days

· Identify weekday or weekend

· Calculate total hours worked

· Apply FLOOR to hours

· Use CASE:

o Consistent Performer if last 7 days AND weekday AND hours ≥ 8

o Irregular Performer if hours < 6

o Absent / Old Record